

ASTRA — SOFTWARE TEAM

3-DAY BUILD CHALLENGE · 2026–27

Build. Solve. Ship.

A recruitment challenge by ASTRA, BMS Institute of Technology & Management — Defence Technology · Software & AI/ML · Electronics & Robotics · Research · Innovation

1. Introduction

ASTRA is a defence-technology-focused student club at BMS Institute of Technology & Management. We build, break, research and ship — across defence technology, software and AI/ML, electronics and robotics.

This is **not a coding interview**. It is a **Technical Build Challenge**.

You will receive **three problem statements** and **exactly three days**. You pick **ONE** and build a working solution. That's it.

We want to see how you:

- understand a problem and break it down
- research independently and use AI tools intelligently
- make reasonable technical decisions under time pressure
- build something functional, debug it, handle edge cases
- document your work and **ship something usable before the deadline**

AI tools are explicitly allowed. Functionality and problem-solving matter more than writing everything from scratch. A student who uses the available tools well and ships a working product is doing exactly what we want.

The challenge is open to students from approximately **1st year to 3rd year**. The bar is deliberately set with a **low entry floor and a high ceiling**: a capable first-year can build a basic working version in three days, while a strong second- or third-year has room to demonstrate significantly deeper work.

2. How the Challenge Works

1. **Choose ONE** of the three challenges: ASTRA INTEL, ASTRA VISION, or ASTRA SENTINEL.
 2. **Build** your solution within **3 days**.
 3. **Test** it properly — including failure cases.
 4. **Document** your work (README, architecture diagram, AI usage disclosure).
 5. **Submit** everything listed in Section 8 via SUBMISSION FORM.
 6. **ASTRA reviews** the submission against the rubric (Section 9).
 7. Shortlisted candidates are invited for a **short technical discussion / demo** of their own project (Section 10) — not a conventional interview.
-

3. Common Rules

3.1 AI IS ALLOWED — AND ENCOURAGED

You may use:

- ChatGPT · Claude · Gemini
- GitHub Copilot · Cursor
- Perplexity
- Any other AI coding or research assistant

AI usage is a productivity tool, and we expect you to use it.

However: you must understand the code you submit and be able to explain your implementation in the technical discussion.

You are responsible for:

- testing AI-generated code
- debugging it
- validating outputs
- understanding your dependencies
- making sure your final solution actually works

Using AI heavily is **not** a disadvantage. Using AI without understanding or validating it **is**.

3.2 TECH STACK IS YOUR CHOICE

Unless a challenge says otherwise, you choose:

- programming languages, frameworks, databases
- ML models, APIs, libraries
- deployment platform

No stack is imposed.

3.3 OTHER RULES

Rule	Detail
Originality	Your submission must be your own work. No copying another candidate's submission.
Open resources	Public/open-source resources, tutorials, datasets, libraries and documentation are allowed. Respect their licenses and give attribution in your README.
Attribution	Credit borrowed code, datasets, designs and models.
Secrets	Never commit API keys, passwords, tokens or <code>.env</code> files containing secrets. Keep them in environment variables.
Data	Use public or self-created data only. Do not use private, confidential or personal data.
Safety	Do not build or deploy harmful, unsafe or surveillance systems against real individuals.
Deadlines	Late submissions are not accepted. Submit before [DEADLINE].

4. Common Starter Resources

ASTRA provides starter resources so every candidate starts from a **common baseline** — and so nobody spends two of their three days just hunting for data.

Full file list: see the internal *Starter Resource Package* document.

4.1 ASTRA Branding Kit

- ASTRA logo (PNG)
- ASTRA logo (SVG)
- White/reversed logo
- Basic brand assets

Use it if you want your project to look like it belongs to ASTRA. Optional.

4.2 ASTRA INTEL

- 2–3 sample defence/technology PDFs
- Example questions
- Sample document metadata

No solution architecture is provided.

4.3 ASTRA VISION

- A small curated starter image dataset
- Clearly labelled folders
- Basic dataset README

Suggested categories: **Aircraft** • **Helicopter** • **Drone** • **Military vehicle** • **Naval**

You may also use your own or publicly available datasets.

4.4 ASTRA SENTINEL

- A small collection of sample public defence/technology articles: title · date · source · URL · content

You may extend it from suitable public sources. **Web scraping is not required.**

5. What ASTRA Provides vs What You Decide

ASTRA provides	You decide
Problem statements	Tech stack
Starter datasets & resources	Architecture
Basic branding assets	Model / API choice
Submission instructions	Database
Evaluation criteria	UI / UX
	Libraries & extra APIs
	Deployment strategy
	Additional features

Everything that affects *how* the solution is built is yours.

6. The Three Challenges

Pick **ONE**. Each is structured as:

- **MUST HAVE** — required for a valid submission (achievable by a capable first-year in 3 days)
 - **SHOULD HAVE** — demonstrates stronger engineering
 - **BONUS** — demonstrates exceptional initiative (never mandatory)
-

CHALLENGE 01 — ASTRA INTEL

AI-Powered Defence Document Intelligence System

Problem

Defence and technology organizations handle large volumes of reports, research papers, news articles and PDFs. Finding relevant information manually is slow.

Build an AI-powered document intelligence system that lets a user upload a document and extract useful information from it.

Example

A user uploads a document about unmanned aerial systems and asks:

"What are the major applications discussed in the document?"

The system should answer using the uploaded document.

Technologies

You may use LLM APIs, open-source models, embedding models, vector databases, RAG, LangChain/LlamaIndex or equivalents, and any suitable frontend/backend framework. **You do not need to train an LLM from scratch.**

MUST HAVE

1. Upload a PDF/document.
2. Extract and process its content.
3. Generate a concise summary.
4. Let the user ask questions about the uploaded document.
5. Answer **based only on the provided document**.
6. Show the relevant section/page the answer came from.

SHOULD HAVE

- Usable interface (upload flow + question flow) with basic error states (corrupt/empty PDF, missing API key, no document uploaded yet).
- Multi-turn conversation with visible history.
- Page-level citations on answers.
- Consistent answers across repeated questions (grounded retrieval, not lucky guessing).

BONUS

- Multiple-document support
 - Semantic search across documents
 - Document comparison
 - Hallucination / unsupported-answer detection
 - OCR for scanned PDFs
 - Relevant-text highlighting
 - Local/open-source models (no paid API)
 - Conversation persistence
-

CHALLENGE 02 — ASTRA VISION

AI-Based Defence Object Recognition System

Problem

Modern defence systems rely on computer vision to identify objects in imagery. Build a computer vision application that identifies or classifies objects in defence-related imagery.

Possible categories: **Fighter aircraft · Helicopters · Tanks · Ships · Military vehicles · Drones**. Choose a limited number of categories instead of attempting everything.

Example

Input: image of an aircraft Output: `Predicted: Fighter Aircraft · Confidence: 87%`

Technologies

Pre-trained models, transfer learning, Hugging Face models, YOLO, OpenCV, TensorFlow/PyTorch, or cloud AI APIs. **You do not need to train a model from scratch** — but you must understand and be able to explain the model you use.

MUST HAVE

1. Accept an image as input.
2. Process it with an ML/CV model.
3. Identify or classify the object.

4. Display the predicted class.
5. Display a confidence score.
6. Give a simple explanation/result to the user.

SHOULD HAVE

- Top-3 predictions with confidences.
- Image preprocessing (resize, normalize, format handling).
- Low-confidence warning ("uncertain prediction").
- Graceful handling of bad inputs (wrong format, huge/corrupt images).
- A short written justification of why this model fits the problem.

BONUS

- Multi-object detection with bounding boxes
- Custom fine-tuning / transfer learning on your own data
- Explainable AI (e.g. saliency/heatmaps)
- Model comparison (two models, same test set, metrics)
- Batch image processing
- Image history / gallery
- Performance metrics (accuracy, precision/recall on a held-out set)

CHALLENGE 03 — ASTRA SENTINEL

AI-Powered Defence Information Monitoring System

Problem

Imagine ASTRA wants to monitor publicly available information related to defence technology. Hundreds of articles, reports and announcements exist.

Build a system that collects, processes, organizes and summarizes defence-related information:

Raw Information → AI Processing → Structured Intelligence

Example

Input: article about a new indigenous UAV program Output: **Category:** Aerospace / UAV ·
AI-generated summary · date · source · **Keywords:** UAV, India, autonomous systems

MUST HAVE

1. Accept a collection of text/articles **or** retrieve from a suitable public source (starter set provided; scraping not required).
2. **Categorize** each item — suggested categories: Aerospace · Naval · Land Systems · Cybersecurity · Space · AI/Robotics · Defence Technology.
3. **Summarize** each item in a short AI-generated summary.
4. **Search** the collected information.
5. **Display** a simple dashboard: article/information · category · date · summary · source.

SHOULD HAVE

- A clean dashboard with category/date filtering.
- A way to add new articles through the UI.
- Data persistence (database or equivalent) instead of in-memory only.
- Keyword extraction per article.
- Error handling for malformed/empty articles.

BONUS

- Automatic topic extraction & entity extraction
- Duplicate article detection and similarity search
- Trend analysis and timeline visualization
- News/article clustering
- RAG-based question answering over the collection — e.g.

"Give me a summary of recent information related to autonomous drones."

producing an **ASTRA Intelligence Brief**: key developments · relevant articles · dates · sources · important entities/topics.

- Topic-based search, source metadata, database persistence

7. Don't Overbuild

You are **not** expected to build a production-grade system in three days.

The order of operations:

1. Build the core functionality.
2. Make it reliable.
3. Test it.
4. Document it.
5. **Then** add advanced features — only if time permits.

A simple working product is better than an ambitious product that does not work.

8. Submission Requirements

Every candidate must submit **ALL** of the following.

8.1 GitHub Repository

The repository must contain:

- Complete source code
- README
- Requirements/dependencies
- Setup instructions
- Environment-variable instructions
- Relevant configuration files
- Dataset instructions, if applicable

Important — your repository must NOT contain:

- API keys
- Passwords
- Tokens

- Private credentials
- `.env` files containing secrets

The submitted repository should be sufficient for a reviewer to understand and, where reasonably possible, run the project.

8.2 Working Demo / Deployed Application

Provide a working deployment wherever reasonably possible:

If deployment is not practical for your project, instead provide:

- Clear local setup instructions
- Required dependencies
- Required environment variables
- The exact commands needed to run the project

A project that cannot be demonstrated must clearly explain why.

8.3 Mandatory Project Explanation Video

Every candidate **must** submit a **detailed project explanation video** (5–8 minutes; slightly longer is acceptable, filler is not).

This is not a promotional video. Its purpose is to show that you understand the problem, your implementation, your architecture, and your technical decisions — and that the application actually works.

Part 1 — Problem understanding (~30–60s)

Which challenge you chose, what problem you solve, why it matters, what your solution aims to achieve.

Part 2 — Live product demonstration (~1–2m)

Actually run the application. Show user interaction, main features, inputs, outputs, at least one realistic example. Screenshots/slides alone are not acceptable.

Part 3 — Complete system architecture (~1–2m) — mandatory

Explain the full flow **input** → **processing** → **AI/ML** → **data layer** → **output** with a clear architecture diagram. Cover frontend, backend, APIs, database, model, data processing, retrieval pipeline (if any), external services and deployment — and **why** you chose this architecture, not just what it is.

```
USER → FRONTEND → BACKEND/API → PROCESSING → AI/ML MODEL  
→ DATABASE / VECTOR DB → RESULT PROCESSING → FRONTEND → USER
```

Your actual architecture will differ. That's expected.

Part 4 — Technical implementation (~1–2m)

The important engineering decisions and system flow: languages, frameworks, APIs, database, model/LLM, embeddings, vector DB, RAG or CV pipeline, data processing, deployment. You do **not** need to explain every line of code.

Part 5 — AI usage (~30–60s)

Which AI tools, which parts were AI-assisted, how, how you validated the output, and what you personally implemented/modified/tested. Example:

"I used Claude to help debug the API integration and generate an initial database schema. I then modified the schema and tested the implementation myself."

AI usage is allowed and will not be penalized. This section is about transparency and understanding.

Part 6 — Challenges & debugging (~30–60s)

One or two significant problems: what went wrong, how you diagnosed it, how you solved it. This section carries a lot of weight for problem-solving ability.

Part 7 — Limitations & future improvements (~30–60s)

Current limitations, known failure cases, what you'd improve, what you'd build with another week.

8.4 Architecture Diagram

The diagram shown in the video must also appear in the README, representing **Input → Processing → AI/ML → Data Layer → Output** where applicable.

Draw.io · Excalidraw · Figma · Canva · Mermaid — anything. It does not need to be visually elaborate: **clarity and technical correctness matter more than design.**

8.5 README

Must contain:

Section	Contents
Project overview	Name, selected challenge, problem being solved
Features	Core + additional
Tech stack	Languages, frameworks, libraries, models, APIs, databases
Architecture	Diagram, component explanation, data flow
Setup	Installation, dependencies, environment variables, run commands
AI/ML	Model(s) used, why chosen, AI pipeline, AI tools used in development
Testing	How tested, example inputs/outputs, known failure cases
Limitations	Current limitations
Future improvements	What you'd add with more time

8.6 AI Usage Disclosure

Include this block in your README:

AI Tools Used:

- ChatGPT
- Claude
- GitHub Copilot

Used For:

- Debugging
- Understanding APIs
- Code suggestions
- Documentation

Major AI-Assisted Components:

- ...
- ...

Personally Implemented / Modified:

- ...
- ...

Validation:

- ...

You are **not** expected to avoid AI. AI usage itself is not evaluated negatively. What matters is whether you can:

Use → Understand → Validate → Improve → Ship

8.7 Final Submission Checklist

Before submitting, verify:

- ☐ GitHub repository submitted
- ☐ Project runs successfully
- ☐ Live demo OR local setup instructions provided
- ☐ README completed
- ☐ Architecture diagram included
- ☐ Detailed project explanation video uploaded
- ☐ Video demonstrates the working project
- ☐ Video explains the complete architecture
- ☐ Video explains major technical decisions
- ☐ AI tools and usage disclosed

- ☐ Limitations documented
- ☐ Future improvements documented
- ☐ No API keys or secrets committed
- ☐ Submission form completed

Important Note to Candidates

The video is an important part of the evaluation.

You are not evaluated only on how impressive the final UI looks. We want to understand:

How did you think about the problem? How did you build the solution? Why did you make your technical choices? How does your system work internally? What went wrong, and how did you fix it? Can you explain and defend what you built?

A technically simple project that is well understood, functional and clearly explained can demonstrate stronger engineering ability than a complex project you cannot explain.

The goal of this challenge is:

**UNDERSTAND → BUILD → TEST → DEBUG
→ EXPLAIN → SHIP**

9. Timeline

Stage	What happens
DAY 0 — Challenge released	Read everything. Choose ONE problem. Set up your repo. Begin.
DAY 1	Build the core MVP. Get end-to-end flow working.
DAY 2	Improve functionality, handle errors, add meaningful features.
DAY 3	Test, document, deploy, record the video, submit.
DEADLINE	All required materials submitted via SUBMISSION FORM by 1st October 2026 .

No extensions. Plan for Day 3 to be for testing and recording — not for new features.

10. Final Message

We are not looking for the person who writes the most code.

We are looking for people who can take a problem, figure things out, use the tools available to them, build something meaningful — and ship it.

Three days. One problem. Your decisions.

BUILD → TEST → IMPROVE → SHIP

See you on the other side. — **ASTRA**